

p0572r0

bit_sizeof and bit_offsetof

Published Proposal, 3 February 2017

This version:

<http://wg21.link/P0572r0>

Author:

[Alex Christensen](#) (Apple)

Audience:

EWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Source:

<https://github.com/achristensen07/papers/blob/master/source/P0572r0.bs>

Abstract

A proposal for the ability to determine where a bit field is stored within a byte.

Table of Contents

- 1 **Introduction and motivation**
- 2 **Behavior of `bit_sizeof` and `bit_offsetof`**
- 3 **`std::bit_size_t`**
- 4 **`bit_offsetof`: macro or a keyword?**

References

Informative References

§ 1. Introduction and motivation

`sizeof` and `offsetof` currently allow programmers to access the layout of data structures with the resolution of one byte. If a data structure needs to be compact, bit fields allow programmers to specify members that take less than one byte, which saves memory but `sizeof` and `offsetof` cannot be used with bit fields. The proposed `bit_sizeof` and `bit_offsetof` keywords would allow code to inspect the locations of individual bits of structures.

In order to interact directly with memory that contains data structures with bit fields, the existing limitations of the C++ language require verbose and error-prone manual memory allocation. Consider the following example program which works in C++14, illustrating the messy and error-prone manual layout of bits necessary so that current C++ programs can know where member variables are stored:

```

#include <string>
#include <limits.h>

struct Foo {
    uint8_t bits; // Memory for A, B, C, and D
    enum class BitSizes {
        A = 2,
        B = 4,
        C = 1,
        D = 1,
    };
    enum class BitOffsets {
        A = 0,
        B = 2,
        C = 6,
        D = 7,
    };
};

static void setBits(uint8_t* address, size_t bitSize, size_t bitOffset) {
    uint8_t bits = UINT8_MAX >> (CHAR_BIT - bitSize);
    *address |= bits << bitOffset;
}

int main() {
    uint8_t memory[2] = {0, 0};
    setBits(&memory[1],
        static_cast<size_t>(Foo::BitSizes::B),
        static_cast<size_t>(Foo::BitOffsets::B));
}

```

The Foo structure should obviously use bit fields because it is many small member variables packed into one byte, but the need to locate the bits in memory currently requires manual allocation of bits in order to access their location. If the `bit_sizeof` and `bit_offsetof` keywords were added to C++, the code would be able to properly use bit fields:

```

struct Foo {
    uint8_t A : 2;
    uint8_t B : 4;
    uint8_t C : 1;
    uint8_t D : 1;
};

int main() {
    uint8_t memory[2] = {0, 0};
    setBits(&memory[1], bit_sizeof(Foo::B), bit_offsetof(Foo, B));
}

```

Finding the locations of members of compact structures is necessary in JIT compilers that interact with data structures where a different instruction must be written into an instruction buffer depending on where the desired bit is located in the destination byte. Development of such a compiler motivated the abandonment of bit fields in a change to WebKit in <https://trac.webkit.org/changeset/166465/trunk/Source/WebCore/rendering/style/RenderStyle.h> and other structures have manual bit allocation for similar reasons. A memory allocator that pre-initializes memory for structures with bit fields would also benefit from knowledge of the locations of bit fields in structures. In general, allowing more precision with bit field location and size determination will enable more efficient code to be written in C++.

§ 2. Behavior of `bit_sizeof` and `bit_offsetof`

`bit_sizeof` is an operator that returns the size in bits of the type of the operand. `bit_offsetof` is an operator that returns the number of bits between the member and the beginning of the structure.

Consider the following illustrative example:

```
struct A {
    uint8_t B : 5;
    uint8_t C : 3;
    uint8_t D;

    static uint16_t StaticMember;
    void Method(){}
    std::size_t operator&(const A&) { return 0; }
};

static void staticFunction() {}
A instance;
struct InheritsFromA : public A {
    uint32_t AnotherMember;
};
InheritsFromA inherits;
A* parentPointer = &inherits;
struct EmptyStruct {};
char fiveChars[5];
```

`bit_sizeof(instance.B)` should return 5. `bit_sizeof(A::C)` should return 3. `bit_sizeof(A::D)` should return `CHAR_BIT` because `bit_sizeof` can be used with members that are not bit fields. `bit_sizeof(A)` should return the number of bits in `A` including padding, similar to [N4296] 5.3.3.2. `bit_sizeof` instance should be a unary expression form corresponding to the unary expression form of `sizeof` in [N4296] 14.6.2.3. [N4296] 5.3.1 defines a `sizeof ... ( identifier )` which counts the number of template parameters in a variadic template, but such a form for `bit_sizeof` would not make sense. Like mentioned in [N4296] 5.1.1.13.3, `bit_sizeof(A::D + 42ull)` should return the size of the result of the contained expression, in this case the number of bits in an `unsigned long long`. Like [N4296] 5.3.3.3, `bit_sizeof(&staticFunction)` should return the number of bits in a function pointer, but `bit_sizeof(staticFunction)` is invalid. Like [N4296] 3.9.1.10, `bit_sizeof(std::nullptr_t)` should be equal to `bit_sizeof(void*)`. Like [N4296] 5.3.3.1, `bit_sizeof(char)`, `bit_sizeof(signed char)`, and `bit_sizeof(unsigned char)` should all equal `CHAR_BIT`. Like [N4296] 5.3.3.2, `bit_sizeof(*parentPointer)` should equal `bit_sizeof(A)`, `bit_sizeof(EmptyStruct)` should be greater than 0, and `bit_sizeof(fiveChars)` should be `5 * CHAR_BIT`. A new definition is necessary linking the definitions `sizeof` and `bit_sizeof`, because `bit_sizeof(uintptr_t)` should be `CHAR_BIT * sizeof(uintptr_t)` and the same should be true for all non-bit-field types. `bit_sizeof(A::Method)` and `bit_sizeof(A::StaticMember)` are invalid like their corresponding `sizeof`. Like [N4296] 5.1.1.5, `class E { int a[bit_sizeof(*this)]; };` should be invalid because it would need to determine the size of an incomplete type.

`bit_offsetof(A, B)` would return 0 if `B` is at the beginning of `A` in memory. `bit_offsetof(A, C)` could return 5 because the beginning of `C` would likely be located 5 bits after the beginning of `A` in memory. `bit_offsetof(A, D)` can work with non-bit-field members and would likely return `CHAR_BITS` depending on how the compiler lays out the members of `A`. A compiler implementer would need to make sure `bit_offsetof` returns the correct offsets with the presence of vtable pointers. Zero-length bit fields cannot be operands of `bit_sizeof` or `bit_offsetof` because they don't have a name, but their presence could influence the values returned by `bit_offsetof` for other members because they change the memory location. Like `offsetof`, `noexcept(bit_offsetof(A, C))` should always be true. Like [N4296] footnote 195, `bit_offsetof` would be required to return the bit offsets even if `operator&` is overloaded. These requirements make less sense for `bit_offsetof` because using `&` or `std::addressof` to get the address of a bit field should still be invalid.

§ 3. `std::bit_size_t`

The return type of `bit_sizeof` and `bit_offsetof` should be specified. I propose one of three options:

1. `std::size_t`. This matches the return type of `sizeof` and `offsetof`, and `std::size_t` is a commonly used type for counting. This presents a problem with large structures. Consider the following code:

```
char largeArray[std::numeric_limits<std::size_t>::max() / CHAR_BIT + 1];
auto overflowIfSizeT = bit_sizeof(largeArray);

struct LargeStruct {
    char LargeArray[std::numeric_limits<std::size_t>::max() / CHAR_BIT + 1];
    char LargeOffset;
};
auto overflowIfSizeT = bit_offsetof(LargeStruct, LargeOffset);
```

If the return type of `bit_sizeof` or `bit_offsetof` were `std::size_t`, then this otherwise valid code would need to be declared to be ill-formed. If someone is iterating all the bits in the entire address space with a `std::size_t`, it will overflow after iterating $1/\text{CHAR_BIT}$ of the bits. This is an existing problem that will be untouched by this specification.

2. A new type `std::bit_size_t` that would be able to hold the maximum value of the number of addressable bits. For example on a 32-bit system with `CHAR_BIT` of 8, a `std::size_t` could be a 32-bit integer because there will never be more than 2^{32} bytes in memory, but a `std::bit_size_t` could be a 64-bit integer so that it can hold the possible maximum value of $2^{40}-1$. As another example, a 64-bit system with a maximum virtual address space size of 2^{48} and `CHAR_BIT` of 8 could use a 64-bit integer for `std::bit_size_t` because it would be able to hold the maximum possible value of $2^{54}-1$. Code would likely often convert between `std::size_t` and `std::bit_size_t`, but on many systems a `static_cast` would not be necessary if they were typedef'd to the same underlying integer type. An implementer might choose to make `std::bit_size_t` the same type as `std::vector<bool>::size_type`.
3. A new strongly typed integer, like a class that has an explicit operator `std::size_t()` or other way to automatically convert to and from `std::size_t`. The explicit would prevent programmers from accidentally converting between the integer types, which could be different. If such a class were created, a corresponding class could be made to wrap a `std::size_t` for converting to the type of integer used for counting bits.

§ 4. `bit_offsetof`: macro or a keyword?

`offsetof` is currently defined to be a macro. Common uses of `offsetof` can be emulated with a macro that subtracts addresses and has no special interaction with the compiler. If `operator&` is overloaded the compiler needs to use something like `std::addressof` which has special behavior in the compiler in order for the `offsetof` macro to behave correctly and comply with footnote 195 of [N4296]. For example, the libc++ implementation of the `offsetof` macro is just `#define offsetof(t, d) __builtin_offsetof(t, d)`

`bit_offsetof` could be specified as a macro to match the definition of `offsetof`, but it would need to have special behavior because there is no way to subtract the addresses of bit fields and because it will also have the condition that it must behave correctly even when `operator&` is overloaded. Because of this need of special behavior, it may be simpler just to define `bit_offsetof` as a new keyword like `bit_sizeof` and `sizeof`.

§ References

§ Informative References

[N4296]

Richard Smith. [Working Draft, Standard for Programming Language C++](https://wg21.link/n4296). 19 November 2014. URL: <https://wg21.link/n4296>